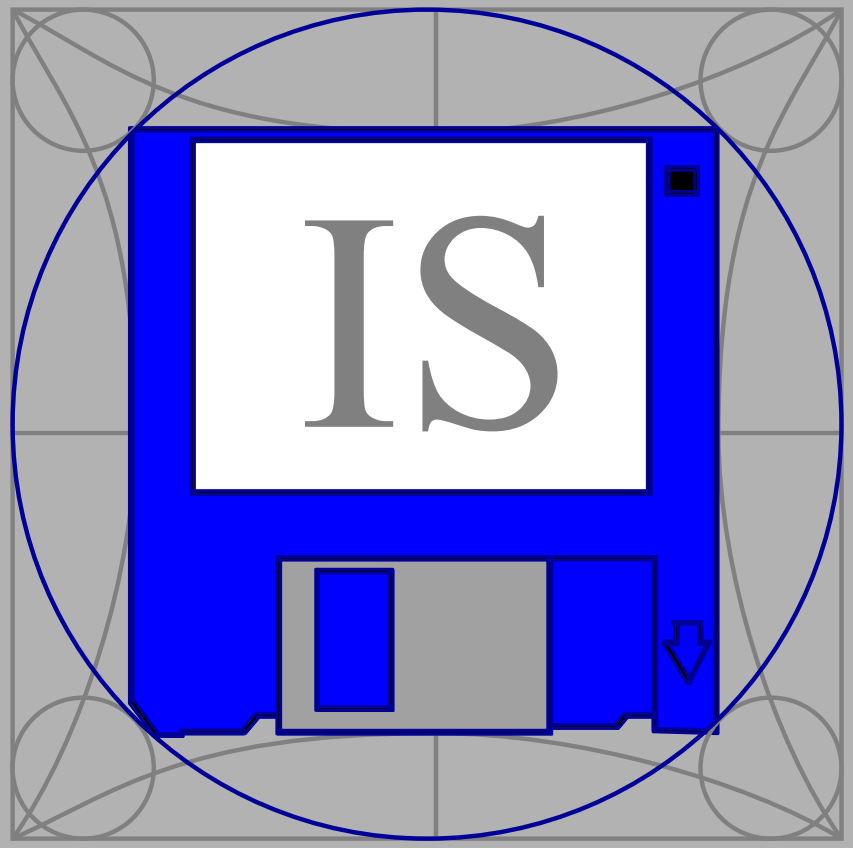


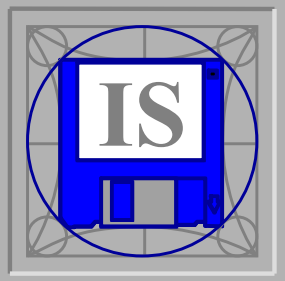
Il ciclo di vita del SW



Ingegneria del Software

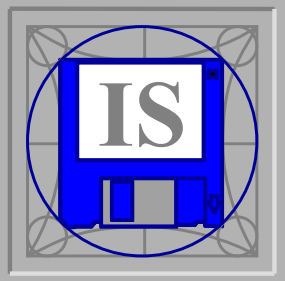
V. Ambriola, G.A. Cignoni,
C. Montangero, L. Semini

Aggiornamenti : T. Vardanega (UniPD)



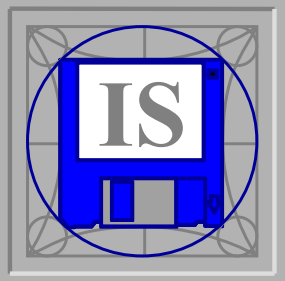
Il concetto di ciclo di vita – 1/2

- Concezione → sviluppo → utilizzo → ritiro
  - I principali stati di vita di un prodotto SW
  - Qui noi ci concentriamo solo sul segmento [concezione → sviluppo]
- La transizione tra stati (l'arco di automa a stati finiti) avviene per azione di processi di ciclo di vita
  - Quelli di cui abbiamo parlato nella lezione precedente
- Compito di un progetto è far progredire lo stato di avanzamento di un prodotto SW lungo il suo ciclo di vita
- Mobilitando specifiche attività di specifici processi
  - Attività ordinate secondo le dipendenze tra loro ingressi e uscite
  - Che anche fissano i criteri di loro attivazione e completamento



Il concetto di ciclo di vita – 2/2

- Chiamiamo «fase» il tempo di stazionamento in un singolo stato di ciclo di vita o in una singola transizione tra stati
  - Uno stato stabile, entro un dato segmento temporale
  - «Fase» non è sinonimo di «attività»
- Esistono svariati cicli di vita, che differiscono per transizioni tra stati e regole di attivazione
  - Per distinguerli, ne astraiamo le caratteristiche in «modelli» di ciclo di vita
- Aderire a un particolare modello di ciclo di vita comporta vincoli sulla pianificazione e gestione del corrispondente progetto
  - Questo influenza il *way of working* e i corrispondenti strumenti di supporto
- In questo corso, ci occupiamo di modelli di sviluppo SW
  - Lo sviluppo è il tratto di ciclo di vita di nostro interesse

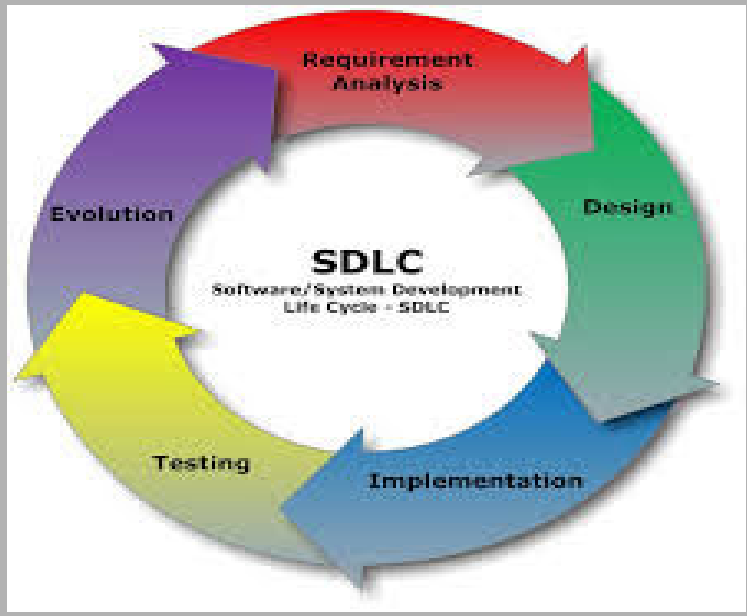


Cosa significa "modello"

- Insieme di specifiche che descrivono un fenomeno di interesse (astratto o concreto)
  - In un modo che non dipende dall'osservatore
  - Dimostrato corretto, empiricamente o per teorema
- I modelli aiutano a studiare, comprendere, misurare, trasformare l'oggetto di interesse
  - Il modello specifica cosa quell'oggetto sia
  - L'architettura (*design*) specifica come esso funzioni
  - L'analisi (del modello) spiega perché quell'oggetto fa quel che fa nel modo in cui lo fa

Esempio

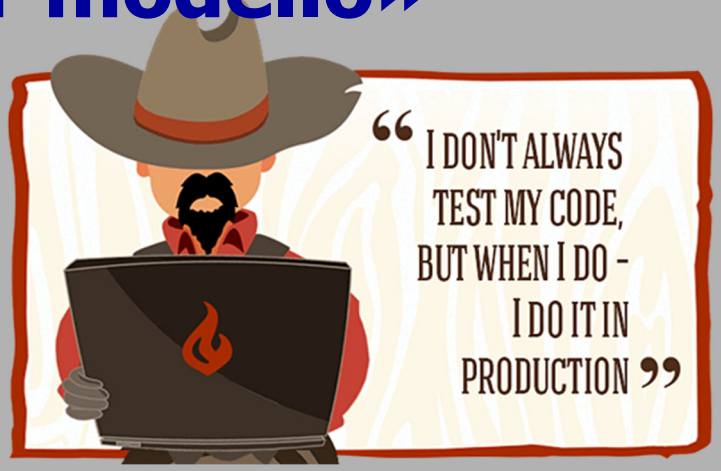
- Un modello di ciclo di vita che <u>non</u> contempla «ritiro» (fine vita)



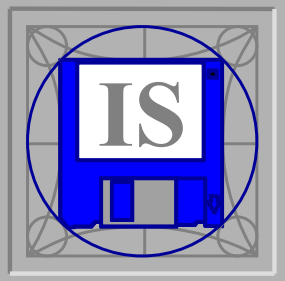
- Lo stadio di «evoluzione» (manutenzione evolutiva) innesca nuovi attraversamenti di ciclo

Modelli di sviluppo – 1/2

- **Alle origini vi fu solo il «non-modello»** ***Code-'n-Fix***
  - **Aka " *Cowboy coding* "**



- **Attività senza organizzazione comprensibile dall'esterno**
  - **L'esatto contrario di sistematico, disciplinato, quantificabile …**



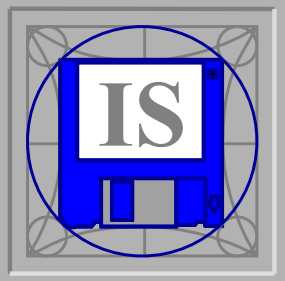
Modelli di sviluppo – 2/2

- Quello stile causò la crisi del SW, che portò alla nascita della disciplina SWE
- Ne nacque una successione di modelli organizzati

Modello	Tratti caratteristici
Cascata	Rigide fasi sequenziali
Incrementale	realizzazione in più passi
A componenti	Orientato al riuso
Agile	Altamente dinamico, fatto di brevi cicli iterativi e incrementali

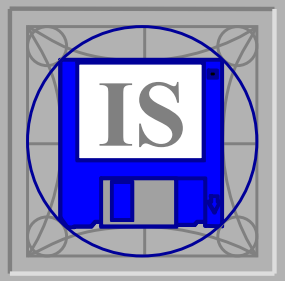
Glossario

- **Iterazione**: raffinamenti o rivisitazione (distruttiva)
  - Come la pittura, che aggiunge (+) ma anche copre (-)
- **Incremento**: aggiunte successive a un impianto base (costruttivo)
  - Come la scultura, che opera solo per sottrazione
- **Prototipo**: provare e scegliere soluzioni
  - Usa-e-getta oppure per produrre incrementi
- **Riuso**
  - Copia-incolla opportunistico, «come capita» (basso costo, scarso impatto)
  - Sistematico per progetto / famiglia di prodotti / ogni prodotto (maggior costo, maggiore impatto)



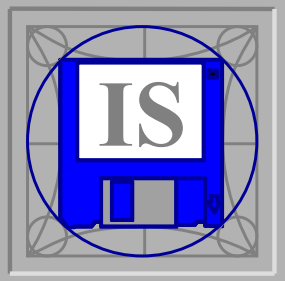
Modello sequenziale (a cascata) – 1/3

- **Definito nel 1970 da Winston W. Royce**
  - "***Managing the development of large software systems: concepts and techniques***"
  - **Centrato sull'idea di processi ripetibili**
- **Successione di stati, rigidamente sequenziale**
  - **Sono fasi, perché sono stati stabili**
  - **Non ammette ritorni a stati precedenti (visti come fallimenti)**
  - **Le iterazioni costano troppo: non sono un buon mezzo di mitigazione delle incertezze di sviluppo**
- **Prodotti**
  - **Principalmente documenti, fino poi a includere il SW**



Modello sequenziale (a cascata) – 2/3

- L'ingresso in uno stato di avanzamento è vincolato da pre-condizioni (*gate*)
  - Che devono essere soddisfatte – dimostrabilmente – dalle post-condizioni delle transizioni in ingresso
- In questo modello, il progetto è una successione di fasi distinte, non sovrapposte
- Usato per lo sviluppo di sistemi complessi, organizzati in «catene di fornitura»
  - E quindi soprattutto per ragioni organizzative



Modello sequenziale (a cascata) – 3/3

- Ogni fase (stato/transizioni) viene definita da
  - Attività previste e prodotti attesi in ingresso e in uscita
  - Contenuti e struttura di documenti che descrivono lo stato raggiunto e le attività svolte
  - Responsabilità e ruoli coinvolti nelle attività
  - Scadenze di consegna dei prodotti
- Entrare, uscire, stazionare in una fase comporta lo svolgimento di determinate azioni
  - Realizzate come attività di specifici processi

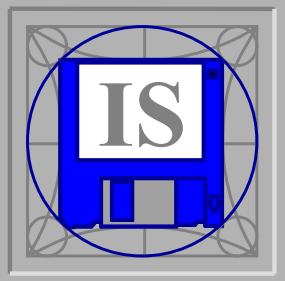
Schema secondo ISO 12207:1995



Alle attività qui elencate se ne aggiungono molte altre istanziate da altri processi (documentazione, verifica, configurazione, gestione di progetto, ...)

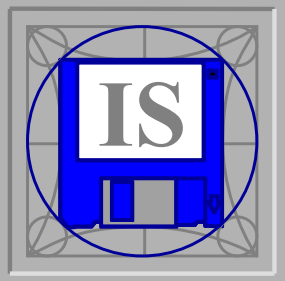
Legenda
AR: analisi dei requisiti
DA: progetto (*design*) architetturale
DD: progetto (*design*) di dettaglio

La manutenzione comporta ritorni sulle fasi precedenti



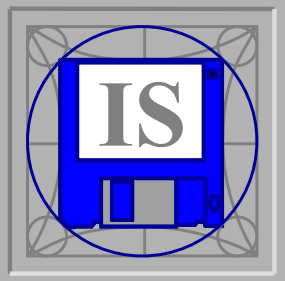
Critica del modello sequenziale

- **Difetto principale: eccessiva rigidità**
  - **Stretta sequenzialità: nessun parallelismo, nessun ritorno**
  - **Non ammette modifiche nei requisiti in corso d'opera**
  - **Visione rigida (burocratica) e poco realistica del progetto**
- **Correttivo 1: con prototipazione**
  - **Prototipi di tipo "usa e getta"**
  - **Per capire meglio i requisiti o le soluzioni, strettamente all'interno di singole fasi**
- **Correttivo 2: con ritorni**
  - **Come «allenamenti» prima dell'atto definitivo**
  - **Per imparare a fare meglio ciò che serve a realizzare il prodotto**



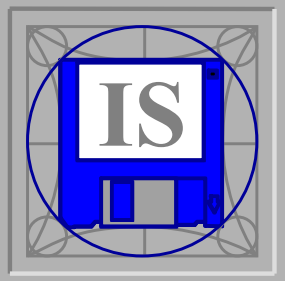
Ritorni: iterazione o incremento?

- Problemi molto complessi richiedono di procedere provando, accettando il rischio di fallimento
  - Questo induce iterazioni potenzialmente distruttive
- Conviene che lo sviluppo proceda per piccoli passi incrementali, che aggiungono, «senza rompere»
  - Evitando di integrare il prodotto tutto-in-una-volta (*big-bang-integration*)
  - Adottando integrazione continua
- Iterazione e incremento coincidono quando la l'aggiunta sostituisce, raffinando, senza impatto negativo sul resto



Vantaggi dei modelli incrementali

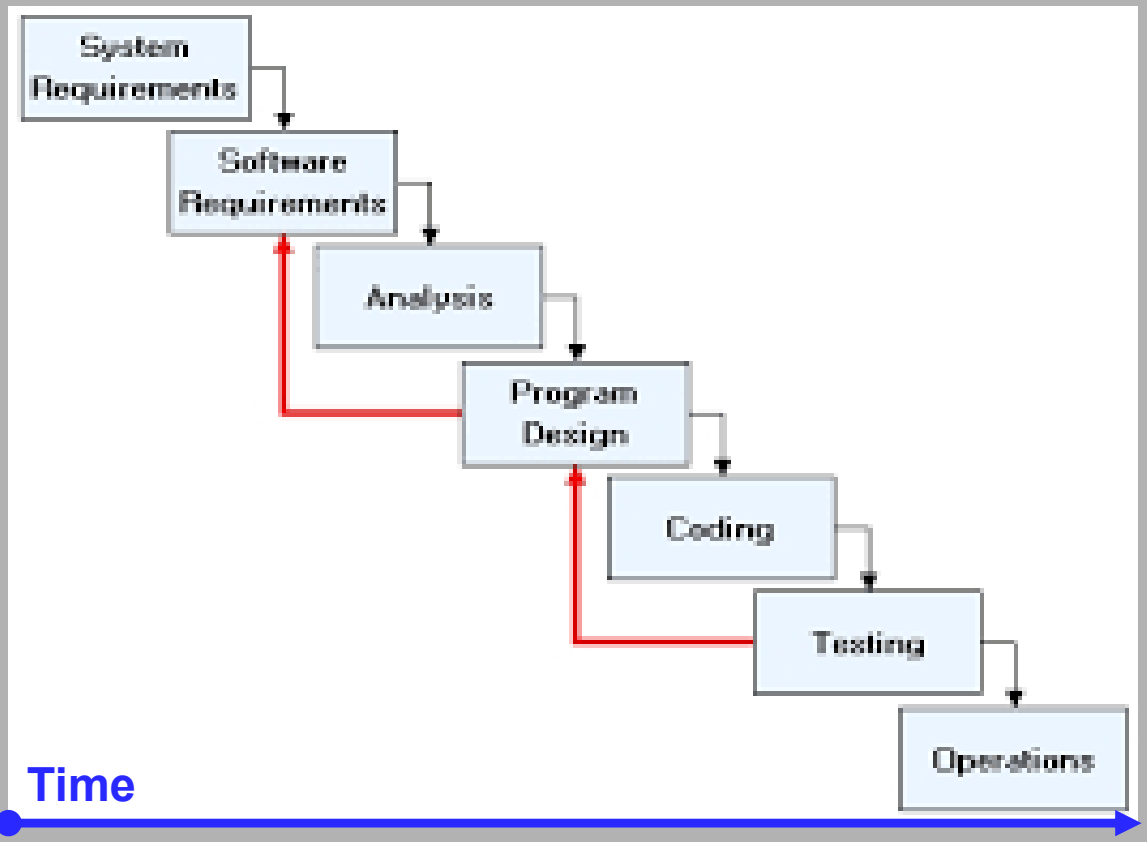
- Possono produrre valore a ogni incremento
  - Un insieme crescente di funzionalità utili diventa presto e progressivamente disponibile
  - Magari a valle di buona prototipazione (non usa-e-getta)
- Procedere per incrementi riduce il rischio di fallimento
  - Anche se il rischio non può essere azzerato …
  - Come un ciclo `for`, da cui sappiamo quando usciremo, a meno di eccezioni
- Le funzionalità fondamentali (quelle più immediatamente necessarie) vanno sviluppate prima
  - Il loro uso frequente aiuta a verificare che siano solide



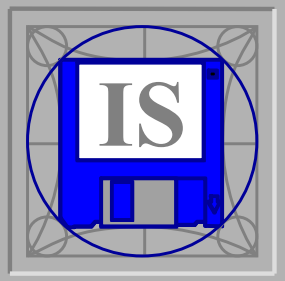
Vantaggi dei modelli iterativi

- Possono applicare a qualunque modello di sviluppo, ma a rischio di romperlo
- Consentono capacità di adattamento a situazioni di crisi
  - Difficoltà tecniche, cambio di requisiti, collasso tecnologico
- Comportano il rischio di non convergenza
  - Come un ciclo `while`, da cui non sappiamo per certo se e quando usciremo
- Tecniche di mitigazione
  - Decomporre il sistema in parti, lavorando prima su quelle più critiche, perché più complesse o con requisiti più incerti
  - Fissando un limite superiore al numero di iterazioni

Rischi dei modelli iterativi – 1/2

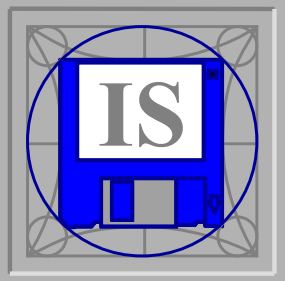


Ogni iterazione comporta un ritorno all'indietro nella direzione opposta all'avanzamento del tempo



Rischi dei modelli iterativi – 2/2

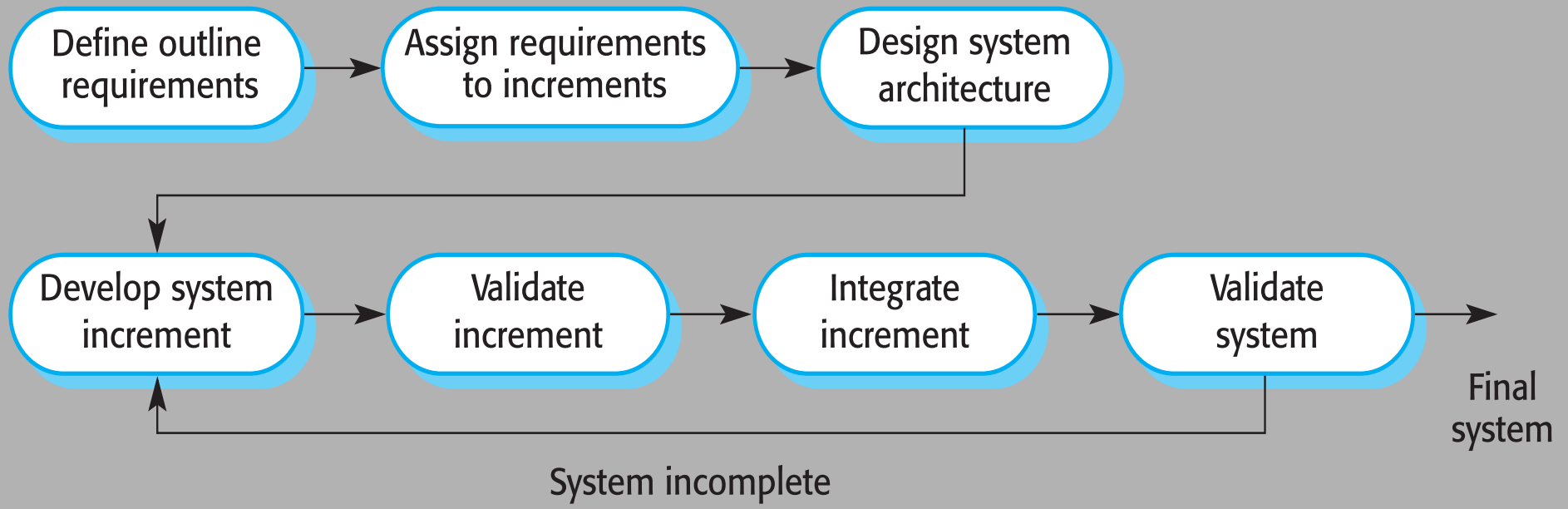
- Nel 1992 si inizia a parlare di *technical debt* per designare parti di *design* o codice bisognose di *refactoring*, cioè di future «passate iterative»
  - Quelle parti costituivano un debito, contratto per avanzare più velocemente, da saldare al più presto, per non pagarlo, dopo, con interessi composti (cioè al costo di cambiamenti invasivi)
- Oggi quel termine designa tutti i punti dello sviluppo nei quali la soluzione realizzata non concorda con la nostra comprensione corrente di come dovrebbe essere
  - Quei tipi di soluzione sono detti *kludge* e sono debiti da sanare presto, perché la loro presenza toglie qualità al prodotto
- Vedere la risorsa «Per approfondire» associata a questo argomento



Modello incrementale – 1/2

- **Prevede rilasci multipli e successivi**
  - **Ciascuno realizza un incremento di funzionalità**
- **I requisiti vanno classificati presto e trattati in base alla loro importanza strategica**
  - **I primi incrementi puntano a soddisfare i requisiti più fondamentali (quelli su cui si poggia il resto)**
    - Funzionalmente (interfaccia utente o persistenza dei dati?)
    - Oppure per necessità strategica (rapporti con gli *stakeholder*)
- **I requisiti più fondamentali diventano chiari e stabili prima, e quindi più facili da soddisfare**
  - **Così quelli meno immediati hanno più tempo per stabilizzarsi e armonizzarsi con lo stato del sistema**

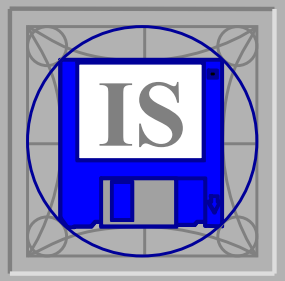
Schema generale



I cicli di incremento sono parte dello sviluppo

La validazione può anch'essa essere incrementale se ogni rilascio è pubblico

Tratto da: Ian Sommerville, *Software Engineering*, 8th ed.



Modello incrementale – 2/2

- **Analisi dei requisiti e progettazione architetturale vengono svolte in una stessa passata iniziale**
  - **Per stabilizzare presto i requisiti principali**
  - **Per stabilizzare presto l'architettura (*design*) complessiva del sistema, che deve essere capace di accogliere incrementi futuri senza rompersi**
  - **Per poter decidere preventivamente il numero di incrementi e i loro specifici obiettivi**

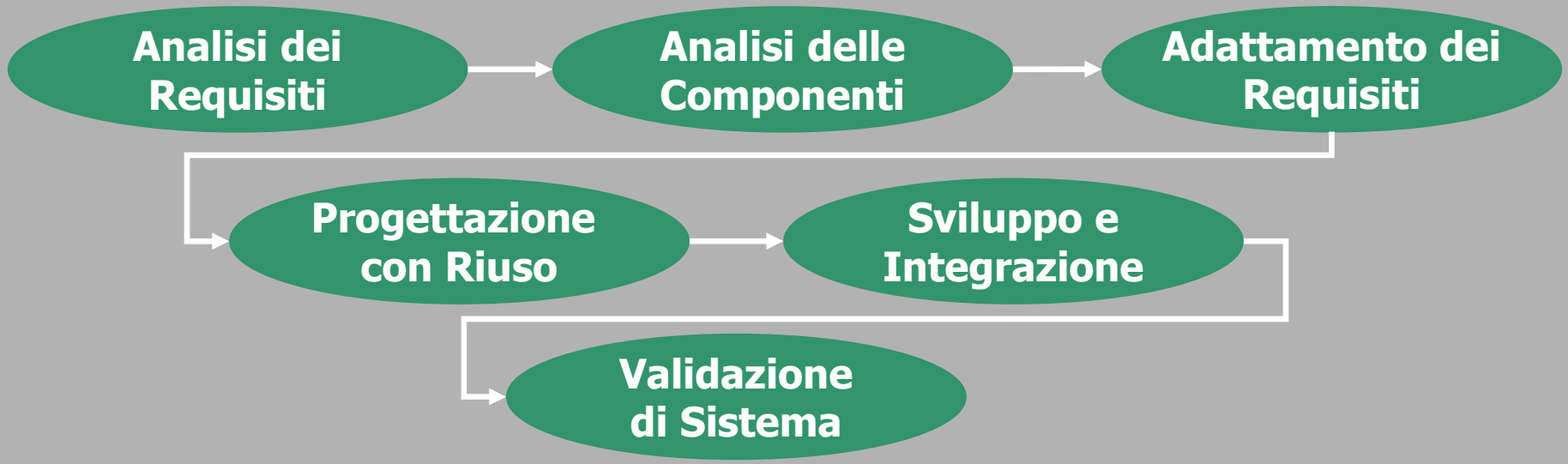
Su questo ritorneremo più avanti

- **La realizzazione avanza per passi incrementali**
  - **Raffinando l'analisi dei requisiti e la progettazione di dettaglio, strettamente entro l'architettura adottata**
  - **Il completamento dei primi incrementi serve a rendere disponibili le principali funzionalità**

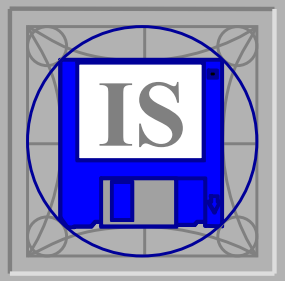
Schema secondo ISO 12207:1995



Modello a componenti

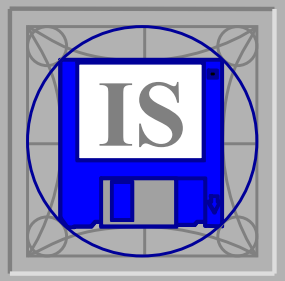


- Molto di quello che ci serve già esiste
- Molto di quello che faremo potrebbe servirci ancora
  - Analisi dei requisiti guidata dalla possibilità di riuso di quanto già esista
  - Realizzazione che cerca di favorire riuso futuro



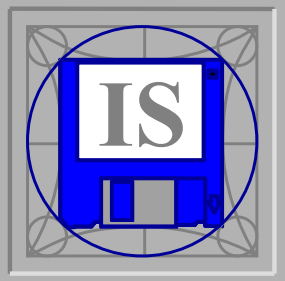
Metodi agili – 1/3

- **Nascono alla fine degli '90 in reazione all'eccessiva rigidità dei modelli allora prevalenti**
  - **http://agilemanifesto.org/**
- **Si basano su quattro principi ideologici**
  - ***Individuals and interactions over processes and tools***
    - L'eccessiva rigidità ostacola l'emergere del valore
  - ***Working sofware over comprehensive documentation***
    - La documentazione non sempre corrisponde a SW funzionante
  - ***Customer collaboration over contract negotiation***
    - L'interazione con gli stakeholder va incentivata e non ingessata
  - ***Responding to change over following a plan***
    - La capacità di adattamento al cambiare delle situazioni è importante



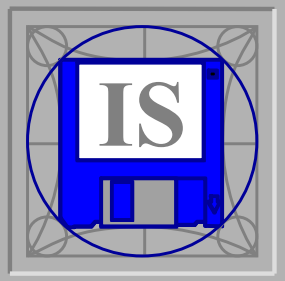
Contro-argomentazioni

- SW privo di documentazione produce costo, non valore
  - Commentare il codice non basta: serve spiegare e motivare le scelte realizzative (il perché precedente)
- Senza un piano, non si possono valutare rischi e direzioni di avanzamento
  - La sola misurazione di consuntivo non basta
- Cambiare si può, ma facendo molta attenzione al rapporto costo/benefici



Metodi agili – 2/3

- L'idea base è il concetto di "*user story*"
  - Una funzionalità significativa che l'utente vuole realizzare con il SW richiesto: uno scenario d'uso
- Ogni "*user story*" è definita da
  - Un documento di descrizione del problema individuato
  - Il verbale delle conversazioni avute con gli *stakeholder* per discutere e comprendere il problema
  - La strategia da usare per confermare che il SW realizzato soddisfi gli obiettivi di quel problema

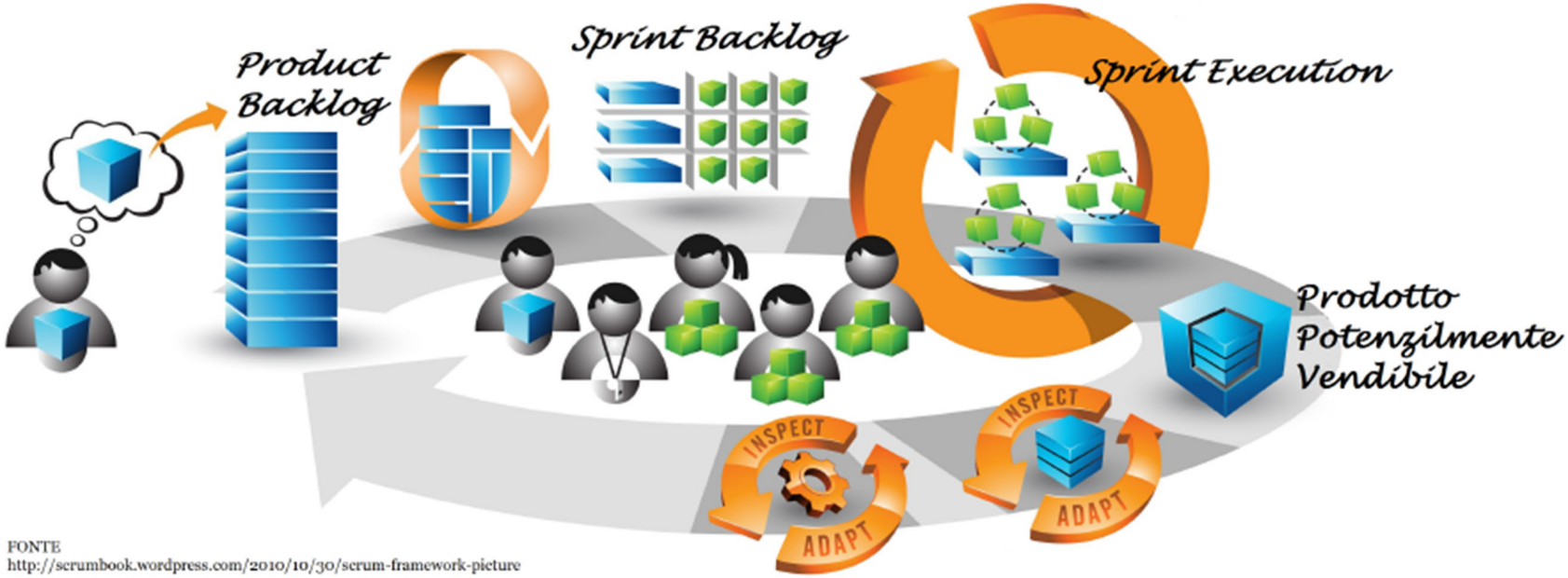


Metodi agili – 3/3

- **Assunti base**
  - Suddividere il lavoro in piccoli incrementi a valore aggiunto, magari anche sviluppabili indipendentemente
  - Sviluppare ciascun incremento in sequenza continua dall'analisi all'integrazione
- **Obiettivi strategici**
  - Poter sempre dimostrare al cliente lo stato corrente
  - Verificare l'avanzamento tramite progresso reale, capace di fornire riscontro (*feedback*)
  - Dare agli sviluppatori la soddisfazione del risultato
- **Buoni esempi**
  - Scrum, Kanban (*just-in-time*), Scrumban

Il vero modello incrementale è il paradiso (culmine ideale) dell'agile

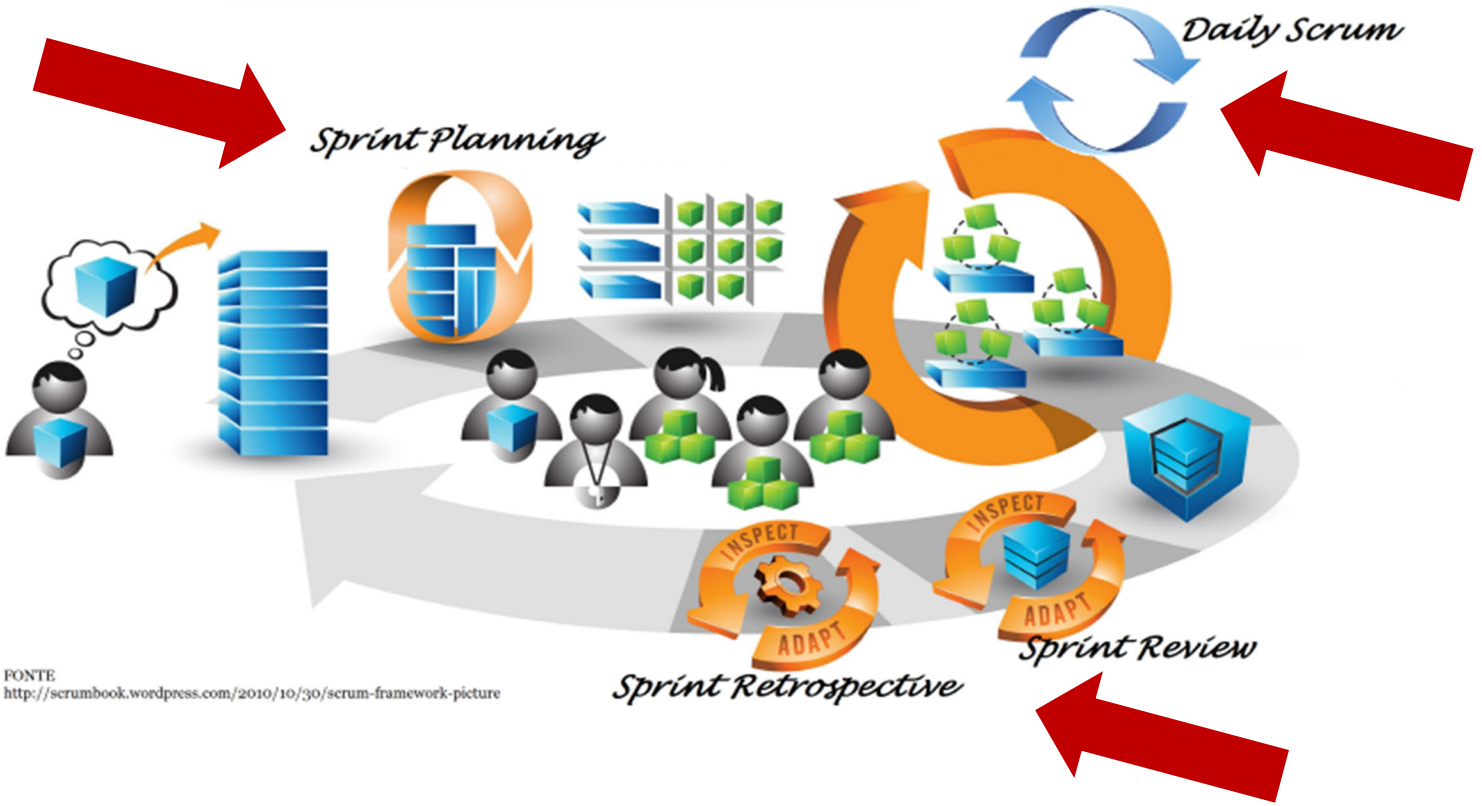
Scrum – 1/2



FONTE
http://scrumbook.wordpress.com/2010/10/30/scrum-framework-picture

- **Product Backlog**
  Requisiti e funzionalità del prodotto
- **Sprint Backlog**
  Insieme di storie del prossimo sprint
- **Sprint**
  Fase operativa di sviluppo
  Durata media 2 - 4 settimane
  Prodotto potenzialmente vendibile

Scrum – 2/2

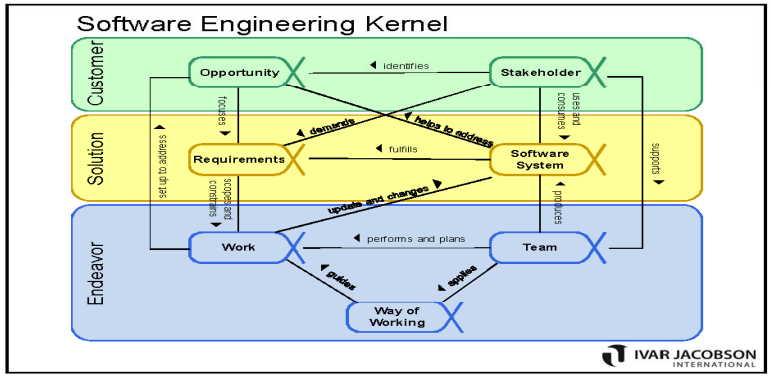


FONTE
http://scrumbook.wordpress.com/2010/10/30/scrum-framework-picture

- **Sprint Planning**
  Pianificazione dello sprint
- **Sprint Review**
  Controllo prodotti dello sprint
- **Daily Scrum**
  Controllo giornaliero avanzamento
- **Sprint Retrospective**
  Controllo qualità sullo sprint

Il ciclo di vita secondo SEMAT – 1/2

IVAR JACOBSON INTERNATIONAL



Stakeholders

- Recognized
- Represented
- Involved
- In Agreement
- Satisfied for Deployment
- Satisfied in Use

The people, groups, or organizations who affect or are affected by a software system.

- Healthy stakeholders represent groups or organizations affected by the software system
- Healthy stakeholder representatives carry out their agreed to responsibilities
- Healthy stakeholder representatives cooperate to reach agreement
- Healthy stakeholders are satisfied with the use of the software system

IVAR JACOBSON INTERNATIONAL

Opportunity

- Identified
- Solution Needed
- Value Established
- Viable
- Addressed
- Benefit Accrued

The set of circumstances that makes it appropriate to develop or change a software system.

- A good opportunity is identified addressing the need for a software-based solution
- A good opportunity has established value
- A good opportunity has a software-based solution that can be produced quickly and cheaply
- A good opportunity creates a tangible benefit

IVAR JACOBSON INTERNATIONAL

Requirements

- Conceived
- Bounded
- Coherent
- Acceptable
- Addressed
- Fulfilled

What the software system must do to address the opportunity and satisfy the stakeholders.

- Good Requirements meets real needs
- Good Requirements has clear scope
- Good Requirements are coherent and well organized
- Good Requirements help drive development

IVAR JACOBSON INTERNATIONAL

www.ivarjacobson.com/semat

Sheet 1 of 2

Il ciclo di vita secondo SEMAT – 2/2

IVAR JACOBSON INTERNATIONAL

Software System

- Architecture Selected
- Demonstrable
- Useable
- Ready
- Operational
- Retired

A system made up of software, hardware, and data that provides its primary value by the execution of the software.

- Good Software System meets requirements
- Good Software System has appropriate architecture
- Good Software System is maintainable, extensible and testable
- Good Software System has low support cost

IVAR JACOBSON INTERNATIONAL

Team

- Seeded
- Formed
- Collaborating
- Performing
- Adjourned

The group of people actively engaged in the development, maintenance, delivery and support of a specific software system.

- A healthy Team meets its team goals effectively
- A healthy Team has members that collaborates effectively
- A healthy Team focus on their work
- A healthy Team continually improves

IVAR JACOBSON INTERNATIONAL

Work

- Initiated
- Prepared
- Started
- Under Control
- Concluded
- Closed

Activity involving mental or physical effort done in order to achieve a result.

- Healthy Work is sizeable, estimate-able and track-able
- Healthy Work breakdown reduces dependencies between work items
- Healthy Work management keeps risks, work and re-work under control

IVAR JACOBSON INTERNATIONAL

Way-of-Working

- Principles Established
- Foundation Established
- In Use
- In Place
- Working Well
- Retired

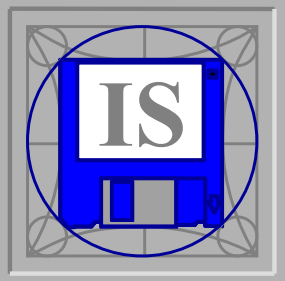
The tailored set of practices and tools used by a team to guide and support their work.

- Good way of working is agreed by the team
- Good way of working reduces risks and technical debts
- Good way of working is effective and removes duplicate work and wastes
- Good way of working improves itself

IVAR JACOBSON INTERNATIONAL

www.ivarjacobson.com/semat

Sheet 2 of 2



Riferimenti

- W.W. Royce, "*Managing the development of large software systems: concepts and techniques*", Atti della conferenza "Wescon '70", agosto 1970
- B.W. Bohem, "*A spiral model of software development and enhancement*", IEEE Software, maggio 1998
- Center for Software Engineering Excellence: https://csee.tech/
- ISO/IEC TR 15271:1998, Information Technology – Guide for ISO/IEC 12207
- Scrum: http://www.scrumalliance.org/